

Kontekst i cilj

Meta-učenje je pristup u strojnom učenju gdje se model trenira na mnogo *sličnih zadataka* kako bi se brzo prilagodio novom zadatku s vrlo malo podataka ¹. **Zero-shot učenje** pak označava situaciju u kojoj model treba generalizirati na potpuno novi zadatak – bez ikakvih primjera tog zadatka u skupu za treniranje. U kontekstu šaha, to znači: **predvidjeti potez u šahovskom otvaranju koje model nikada nije vidio tijekom treninga** (niti jedan primjer otvorenja nije korišten prilikom treniranja). Cilj je da meta-učenje nauči apstrahirati strukture i ideje otvaranja (patterne ljudskih poteza), umjesto da samo memorizira poteze iz baze.

Naš projekt cilja koristiti meta-učenje za **zero-shot predikciju sljedećeg poteza u šahovskom otvaranju**. To znači da ćemo: a) definirati svaki otvoreni zadatak (pojedino otvaranje ili varijantu) kao zasebni „task“ za meta-učenje, b) trenirati model na velikom broju raznih otvaranja, i c) testirati ga na otvaranjima koja smo isključili iz treninga. Evaluirat ćemo koliko dobro model može predvidjeti ljudske poteze u tom neviđenom otvorenju, uspoređujući predikciju s pravim ljudskim potezima. Otvoreni podaci iz Lichess baze (npr. *Lichess Elite Database* ili odgovarajući skelet otvaranja ²) osigurat će skup partija za učenje i evaluaciju.

Meta-učenje i few-shot učenje

Meta-učenje obuhvaća metode koje optimiziraju proces učenja samog modela. Cilj je postići da se model brzo prilagodi novim zadacima s malo primjera. Kao što navode Tang i sur. (2024), few-shot učenje je usko povezano s meta-učenjem ¹. Few-shot se fokusira na generalizaciju iz vrlo ograničenog skupa podataka, dok meta-učenje trenira model na mnogim sličnim zadacima kako bi model naučio *kako učiti* te se bolje prilagođavao novim zadacima. U literaturi se meta-učenje često dijeli na tri tipa: **metričke metode** (npr. prototipne mreže ³), **model-based** pristupe (npr. interne mehanizme koji brzo adaptiraju model) i **optimizacijske metode** (npr. MAML – “Model-Agnostic Meta-Learning” ³). MAML trenira početne težine modela tako da se one mogu brže prilagoditi novom zadatku s nekoliko koraka učenja ⁴.

U šahovskom kontekstu, zadatak *predviđanja ljudskih poteza* također se uklapa u ovo okruženje few-shot učenja – cilj je generalizirati na neviđenog “demošatora” (otvaranje) na osnovi ograničenih podataka (npr. samo nekoliko partija tog otvorenja). Pristup s meta-učenjem za šah relativno je nov, ali postoje srodni radovi: npr. Tang i suradnici (2025) razvili su model *Maia4All* koji fine-tuningom i prototip-mrežom efikasno modelira igrača iz samo 20 partija, u odnosu na 5.000 prije ⁵. Ovakav način inicijalizacije (odabir “prototipnih” zadataka) mogao bi poslužiti i za otvaranja: npr. učiti model na skupovima čestih otvaranja, a zatim fina-tjunati za novonastalo otvaranje. Također, student Lennart August (2023) pokazuje da se s few-shot embeddingom na 100 partija mogu kvalitetno predvidjeti potezi pojedinog igrača ⁶. Ovaj rad ilustrira da je moguće (u ograničenim uvjetima) naučiti reprezentacije koje hvataju stil igre i koristiti ih za predikciju poteza. Analogno, meta-učenje za otvaranja težilo bi naučiti reprezentacije “ideja otvaranja” koje generaliziraju i na nove varijante.

Podaci i reprezentacija poteza

Za ulazne podatke možemo koristiti dostupne šahovske baze (npr. Lichess baze partija). Posebno je koristan **Lichess otvaranja** dataset koji sadrži 3627 šahovskih otvaranja s ECO klasifikacijom ². Svaki unos sadrži niz poteza koji definiraju otvoreno otvaranje, u formatu PGN ili UCI, i pripadajuću početnu poziciju (FEN/EPD) nakon tih poteza ⁷. Primjerice, jedan zapis u datasetu navodi PGN niz poteza i odgovarajuću FEN reprezentaciju pozicije na kraju tog niza ⁷. Takva struktura nam omogućuje da modelu izložimo doslovni raspored figura i poteza do neke ključne pozicije u otvaranju te da za sljedeći potez predvidimo koji će ga igrač odabrati.

Reprezentacija pozicije može se napraviti na standardne načine: npr. **FEN niz** možemo kodirati u binarni tensor dimenzija $8 \times 8 \times (12 + X)$ (12 kanala za različite tipove figura i boju, plus eventualno dodatni kanali za stranu na potezu i rokadu). Alternativno, poput nekih modela, moguće je tretirati sekvencu poteza do pozicije kao tekstualni niz tokena (npr. UCI ili SAN) i trenirati Transformer na toj sekvenci. U svakom slučaju, izlaz je klasifikacija sljedećeg poteza (odabir među svim legalnim potezima). Strukturu modela možemo uzeti sličnu radu Tang i sur. (2024) – npr. Transformer arhitekturu prilagođenu za ljudski stil (kao Maia-2) ⁸. Za praktičnu implementaciju iskoristimo biblioteku `python-chess` za parsiranje PGN-a i generiranje FEN-ova iz partija. Otvorenije baze poput Lichess omogućuju skidanje velikog broja partija filtriranih po razini (ELO) i varijantama otvaranja.

Za izgradnju zadataka kod meta-učenja, možemo svakom ECO kodu ili nazivu otvaranja pridružiti task. Npr. zadatak može biti „predviđanje poteza u otvorenju Sicilijanska obrana Najdorf-Varijanta“. Tijekom meta-treninga model će promatrati mnoge otvaranja (tasks), a za svaki task dobit će manji skup primjeraka (npr. 5–20 partija iz tog otvorenja). Na meta-testu će dobiti posve novo otvorenje (task) kojeg nije vidio – takve “zero-shot” zadatke koristimo samo za evaluaciju.

Metrike i baseline metode

Za evaluaciju možemo koristiti mjere slične onima u radu *Grandmaster-Level Chess Without Search* ⁹, gdje je **action-accuracy** definirana kao postotak stanja u kojima model predvidi točno *ground-truth* potez ⁹. Npr. Tang i suradnici (2024) pokazuju da veliki transformer model sa 270M parametara postiže ~93.5% akcijske točnosti predviđajući Stockfishov najbolji potez ¹⁰. U našem slučaju, umjesto najboljih poteza koristimo stvarne ljudske poteze. Također možemo koristiti **Kendall-tau korelaciju** da usporedimo poredak svih predloženih poteza sa stvarnim poretком preferencija igrača ⁹. Kao baseline mogu poslužiti jednostavni modeli: npr. model koji se trenira na kombiniranim podacima svih trening otvaranja bez posebnog prilagođavanja (tj. standardna jednozadana mreža). Također možemo usporediti metodu meta-učenja s čistim “transfer learningom”: treningom na svim dostupnim partijama i testiranjem na neviđenom otvorenju. Jednako tako, može se koristiti i klasična statička analiza (npr. Stockfish ili prosječni move-stats), ali važno je napomenuti kako takvi engine-i ne modeliraju ljudsko ponašanje, već traže optimalni potez.

Preporučeni pristup i metodologija

1. **Priprema podataka:** Preuzeti Lichess partije i izdvojiti skupove partija po otvorenjima (npr. prema ECO kodovima). Naći otvaranja s dovoljnim brojem partija (npr. barem 50–100) za trening. Podijeliti ih na metazadatke (tasks). Primjer: ECO kod “B20” (Sicilijanska obrana) može biti jedan task, “C50” (talijanska) drugi, itd. Ukloniti jedno ili više otvorenja iz trening skupa za kasniju zero-shot evaluaciju.

2. **Model i reprezentacija:** Definirati neuralnu mrežu za predviđanje poteza. Može se koristiti **Transformer** (poput Maia-2 ⁸) koji prima ulaznu poziciju (FEN ili sekvencu poteza) i vraća vjerojatnosti za svaki legalni potez. Alternativno, LSTM/GRU ili čak CNN nad tablom mogu poslužiti. Izlaz modela neka bude distribucija preko svih mogućih poteza (kategorizacija).
3. **Meta-učenje:** Primijeniti algoritam meta-učenja. Na primjer, **MAML (Model-Agnostic Meta-Learning)** trenira mrežu na način da njezine početne težine budu optimalne za brzo prilagođavanje novom tasku sa samo nekoliko koraka gradijenta. Ovaj pristup se izravno spominje u literaturi kao optimizacijski meta-algoritam ⁴. U alternativni se mogu razmotriti metrički pristupi: npr. prototipna mreža (engl. *Prototypical Networks*) gdje se u embedder prostor projectiraju pozicije i kreiraju se prototipni centri za tipične poteze u svakom opening-u ³. (U kombinaciji, mogli bismo primijeniti dva koraka: prvo trenirati "meta-model" na uobičajenim otvorenjima, a zatim prilagoditi na nekoliko partija novog otvaranja.) Maia4All pristup prototipa pokazuje da je korisno prvo pretrenirati na skup *prototipnih igrača*, pa zatim fina-tune na ciljanim sa malim podacima ⁵ – analogno tomu, mogli bismo prvo učiti na reprezentativnim otvorenjima s mnogo podataka, a zatim prilagoditi model novom otvorenju.
4. **Trening i evaluacija:** Tijekom meta-treninga koristiti ciklus epizoda: za svaki zadatak (otvaranje) uzeti N primjera (partija) za unutarnji loop učenja i dodatne za vanjski loop optimizacije. Čuvati model s najboljom meta-generalizacijom (ili meta-bazu). Za test, modelu dati N partija novog otvaranja i promjeriti njegovu predikciju na sljedećem potezu. Mjeriti *action-accuracy* i usporediti s ljudskim potezima. Evaluirati i baseline modele (bez meta-učenja) radi usporedbe.
5. **Baseline sustavi:** Kao jednodušnu točku usporedbe može se uzeti model treniran na svim otvaranjima bez meta-učenja. Također, usporediti s jednostavnom strategijom (npr. najčešći potez u datom otvorenju).
6. **Alati i knjižnice:** Koristiti Python 3, biblioteku **PyTorch** za model i učenje, **learn2learn** ili vlastitu implementaciju MAML-a za meta-trening, i **python-chess** za rad s PGN/FEN.

Zaključak i preporuka

Kombinirajući sve navedeno, preporučujemo pristup u kojem se model trenira meta-učenjem na različitim otvorenjima kako bi se postigla generalizacija na neviđena otvaranja. Očekujemo da meta-učenje omogući prepoznavanje obrazaca u otvaranjima (npr. tipičnih struktura figura) koje model *prenosi* na novo otvaranje. Pritom je ključno pažljivo definiran set zadataka i razmjer podataka (npr. sve igre do 10. poteza). U konačnici, nakon što model „nauči učiti“, možemo generirati prompt za **Codex** (ili sličan generator koda) koji implementira ovaj pipeline: obrada podataka, definiranje modela i meta-treninga, te evaluaciju.

Primjer prompta za Codex

```
Write a Python project that implements a meta-learning solution for zero-shot move prediction in chess openings.
```

```
GOAL:
```

```
- Predict the next move in a chess opening that was not seen during training.
```

```
DATA:
```

```
- Use the Lichess openings dataset (e.g. via Kaggle or HuggingFace) or Lichess database of games.
```

```
- Each game provides a sequence of opening moves (in PGN or UCI) and resulting board position (FEN).
```

- Organize tasks by opening (ECO code or opening name). Reserve at least one opening entirely for zero-shot test.

TECHNOLOGY:

- Python 3.11
- PyTorch (or TensorFlow) for modeling.
- `python-chess` or similar to parse PGN and extract board states (FEN).
- Optionally `learn2learn` library for meta-learning algorithms.

MODEL ARCHITECTURE:

- A neural network (e.g. Transformer or LSTM) that takes the current board (or move sequence) as input and outputs a probability distribution over legal next moves.
- The network should be capable of quickly adapting to a new opening with few examples.
- Use a meta-learning algorithm (e.g. MAML or ProtoNet):
 - For MAML: implement inner-loop and outer-loop updates so the model can fine-tune on a few games from a new opening.
 - Alternatively, use a prototypical network approach to embed positions or whole-game contexts and compute prototypes for openings.

TRAINING:

- Preprocess games to extract opening segments (e.g. first 5-10 moves) and label each move.
- Meta-train: in each episode, sample an opening (task), use a small support set of games to adapt the model, then test on query games from the same opening.
- Meta-objective: minimize prediction loss on query games after adaptation.

BASELINE:

- Include a baseline where a model is trained normally (without meta-learning) on all training openings.

EVALUATION:

- After training, test on the held-out "zero-shot" opening: give the model a few example games and evaluate its next-move predictions.
- Report metrics such as Top-1 accuracy of predicted moves (percentage of moves correctly predicted) and cross-entropy loss.
- Compare the meta-learning model to the baseline.

OUTPUT:

- Provide complete Python code: data loading, preprocessing, model definition, training loops, and evaluation.
- Include comments and instructions on how to run training and evaluation.

This prompt specifies the task, data, model, and evaluation requirements clearly, enabling Codex to generate a working implementation from one shot.

2 7 Lichess/chess-openings · Datasets at Hugging Face

<https://huggingface.co/datasets/Lichess/chess-openings>

6 fse.studenttheses.ub.rug.nl

<https://fse.studenttheses.ub.rug.nl/34065/1/BP-Lennart-August-s4800036.pdf>

9 10 Grandmaster-Level Chess Without Search

<https://arxiv.org/html/2402.04494v1>